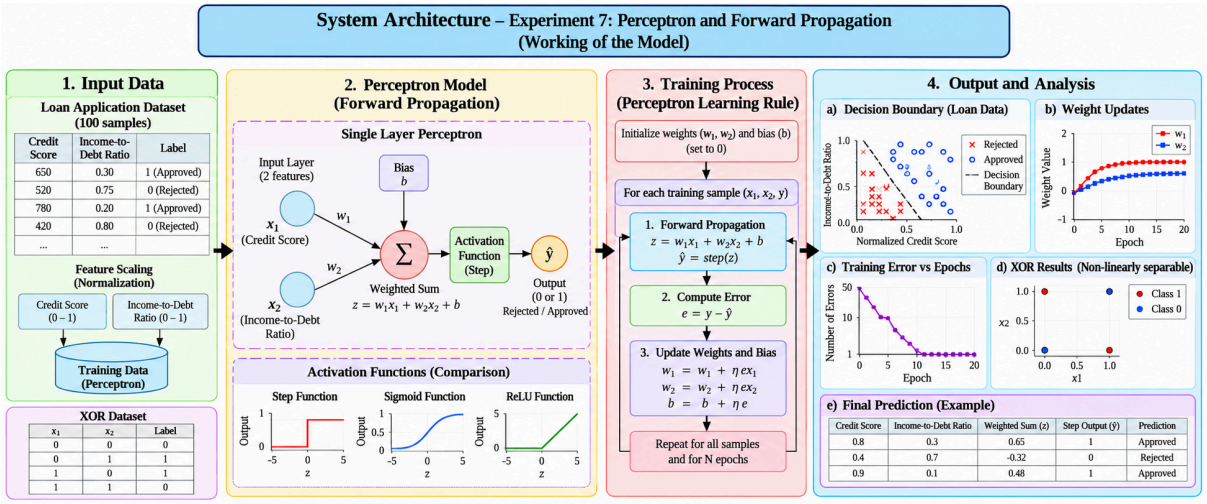
Experiment 7: Perceptron and Forward Propagation

1. AIM

To implement a single-layer perceptron using NumPy and understand forward propagation through weighted sums, bias, and step activation. To train the perceptron on loan approval data, record weight updates during training, visualize the learned decision boundary, demonstrate why a single perceptron cannot solve the XOR problem, and compare step, sigmoid, and ReLU activation functions using Matplotlib.



2. MATHEMATICAL FORMULAS

1. Weighted Sum

$$\boxed{z = \sum_{i=1}^{n} w_i x_i + b}$$

2. Step Activation Function

$$\boxed{f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}}$$

3. Perceptron Prediction

$$\boxed{\hat{y} = f(w^T x + b)}$$

4. Perceptron Learning Rule

$$\boxed{w_i^{new} = w_i^{old} + \eta(y - \hat{y})x_i}$$

Bias update:

$$\boxed{b^{new} = b^{old} + \eta(y - \hat{y})}$$

where η is the learning rate, y is the actual output, and $\hat{y}$ is the predicted output.

5. Sigmoid Activation Function

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

6. ReLU Activation Function

$$\boxed{\text{ReLU}(z) = \max(0, z)}$$

7. Mean Squared Error

$$\boxed{\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

CODE

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# ========================================
# PART 1: GENERATE LOAN APPROVAL DATA
# ========================================

n = 100

# Feature 1: Credit score (300-850)
credit_score = np.random.uniform(300, 850, n)

# Feature 2: Income-to-debt ratio (0-1)
income_debt_ratio = np.random.uniform(0, 1, n)

# Normalize features to [0, 1]
X = np.column_stack([
    (credit_score - 300) / 550,
    income_debt_ratio
])

# Generate labels using a linear decision rule
# 1 = Approved, 0 = Rejected
y = (
    0.7 * X[:, 0] + 0.3 * X[:, 1] >= 0.5
).astype(int)

print("Number of applications:", n)
print("Approved:", np.sum(y == 1))
print("Rejected:", np.sum(y == 0))

# ========================================
# PART 2: PERCEPTRON IMPLEMENTATION
# ========================================

weights = np.zeros(2)
bias = 0.0
learning_rate = 0.1
epochs = 20

weight_history = []
epoch_errors = []

def step_activation(z):
    return (z >= 0).astype(int)

for epoch in range(epochs):
    errors = 0

    for i in range(len(X)):
        # Forward propagation
        z = np.dot(X[i], weights) + bias
        prediction = int(step_activation(z))

        # Perceptron learning rule
```

```
        error = y[i] - prediction

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

        if error != 0:
            errors += 1

    weight_history.append(weights.copy())
    epoch_errors.append(errors)

    print(
        f"Epoch {epoch + 1:2d}: "
        f"Weights = {weights.round(4)}, "
        f"Bias = {bias:.4f}, Errors = {errors}"
    )

# Final training accuracy
predictions = step_activation(X @ weights + bias)
accuracy = np.mean(predictions == y) * 100

print("\nFinal Weights:", weights)
print("Final Bias:", bias)
print("Training Accuracy:", round(accuracy, 2), "%")

# ==========================================
# PART 3: PLOT WEIGHT UPDATES
# ==========================================

weight_history = np.array(weight_history)

plt.figure(figsize=(8, 5))
plt.plot(weight_history[:, 0], marker="o", label="Weight 1")
plt.plot(weight_history[:, 1], marker="s", label="Weight 2")

plt.xlabel("Epoch")
plt.ylabel("Weight Value")
plt.title("Perceptron Weight Updates")
plt.legend()
plt.grid(True)
plt.show()

# ==========================================
# PART 4: PLOT TRAINING ERRORS
# ==========================================

plt.figure(figsize=(8, 5))
plt.plot(
    range(1, epochs + 1),
    epoch_errors,
    marker="o"
)

plt.xlabel("Epoch")
plt.ylabel("Number of Misclassifications")
plt.title("Perceptron Learning Progress")
plt.grid(True)
plt.show()
```

```
# ========================================
# PART 5: DECISION BOUNDARY
# ========================================

plt.figure(figsize=(8, 6))

plt.scatter(
    X[y == 0, 0], X[y == 0, 1],
    label="Rejected", marker="x"
)
plt.scatter(
    X[y == 1, 0], X[y == 1, 1],
    label="Approved", marker="o"
)

x_values = np.linspace(0, 1, 100)

if abs(weights[1]) > 1e-10:
    y_values = -(weights[0] * x_values + bias) / weights[1]
    plt.plot(x_values, y_values, "k--", label="Decision Boundary")

plt.xlabel("Normalized Credit Score")
plt.ylabel("Income-to-Debt Ratio")
plt.title("Loan Approval Perceptron Decision Boundary")
plt.ylim(-0.1, 1.1)
plt.legend()
plt.grid(True)
plt.show()

# ========================================
# PART 6: TEST THE XOR DATASET
# ========================================

X_xor = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y_xor = np.array([0, 1, 1, 0])

w_xor = np.zeros(2)
b_xor = 0.0
lr = 0.1

xor_errors = []

for epoch in range(30):
    errors = 0

    for i in range(4):
        pred = int(step_activation(
            np.dot(X_xor[i], w_xor) + b_xor
        ))

        error = y_xor[i] - pred
```

```
        w_xor += lr * error * X_xor[i]
        b_xor += lr * error

        if error != 0:
            errors += 1

    xor_errors.append(errors)

xor_predictions = step_activation(X_xor @ w_xor + b_xor)

print("\nXOR Inputs:")
print(X_xor)
print("Actual XOR Outputs:", y_xor)
print("Perceptron Predictions:", xor_predictions)
print("XOR Accuracy:", np.mean(xor_predictions == y_xor) * 100, "%")

plt.figure(figsize=(8, 5))
plt.plot(range(1, 31), xor_errors, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Number of Errors")
plt.title("Single Perceptron on XOR")
plt.grid(True)
plt.show()

# ========================================
# PART 7: COMPARE ACTIVATION FUNCTIONS
# ========================================

z = np.linspace(-5, 5, 300)

step = (z >= 0).astype(float)
sigmoid = 1 / (1 + np.exp(-z))
relu = np.maximum(0, z)

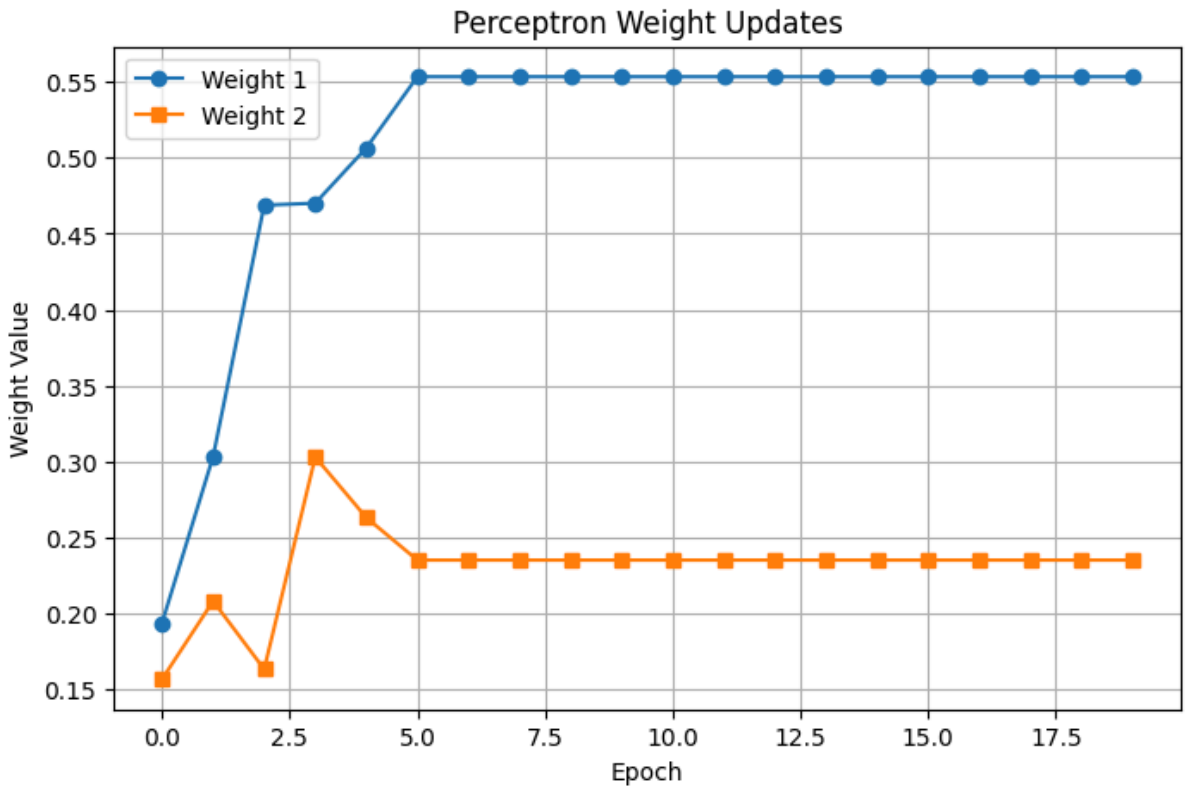
plt.figure(figsize=(9, 5))
plt.plot(z, step, label="Step")
plt.plot(z, sigmoid, label="Sigmoid")
plt.plot(z, relu, label="ReLU")

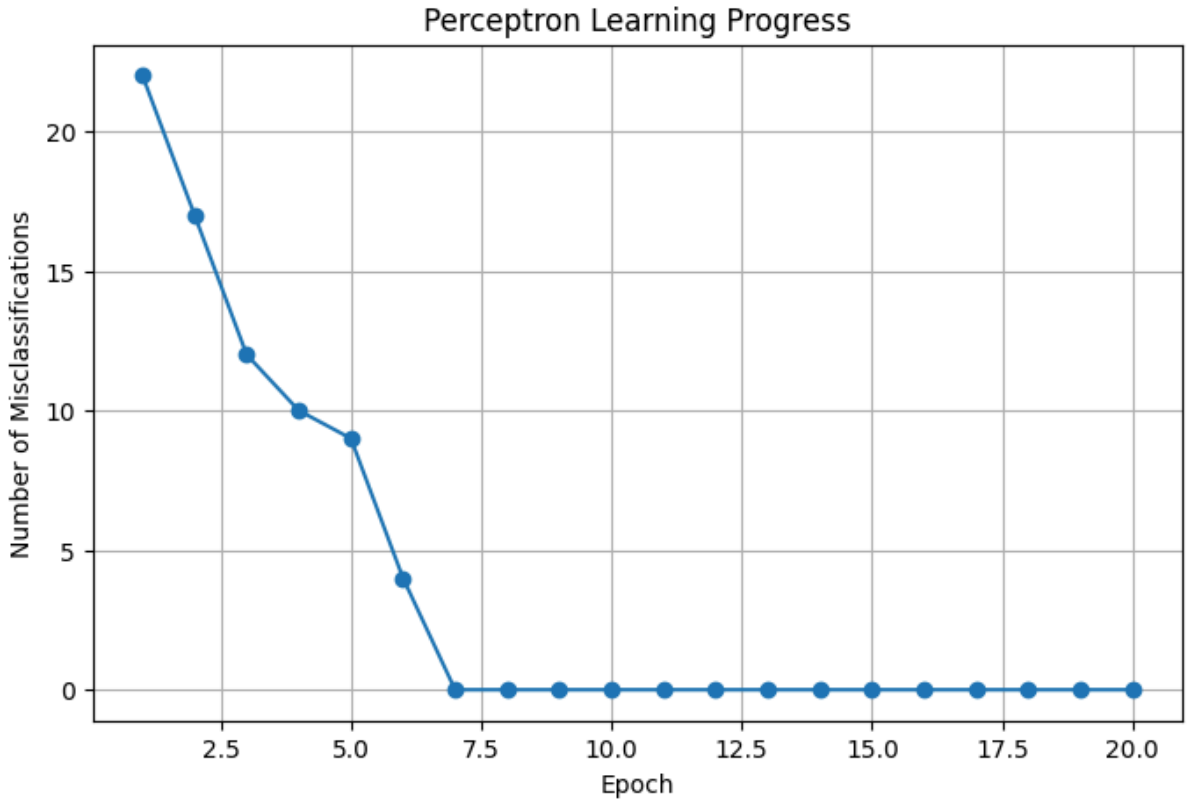
plt.xlabel("Weighted Sum (z)")
plt.ylabel("Activation Output")
plt.title("Comparison of Activation Functions")
plt.legend()
plt.grid(True)
plt.show()
```

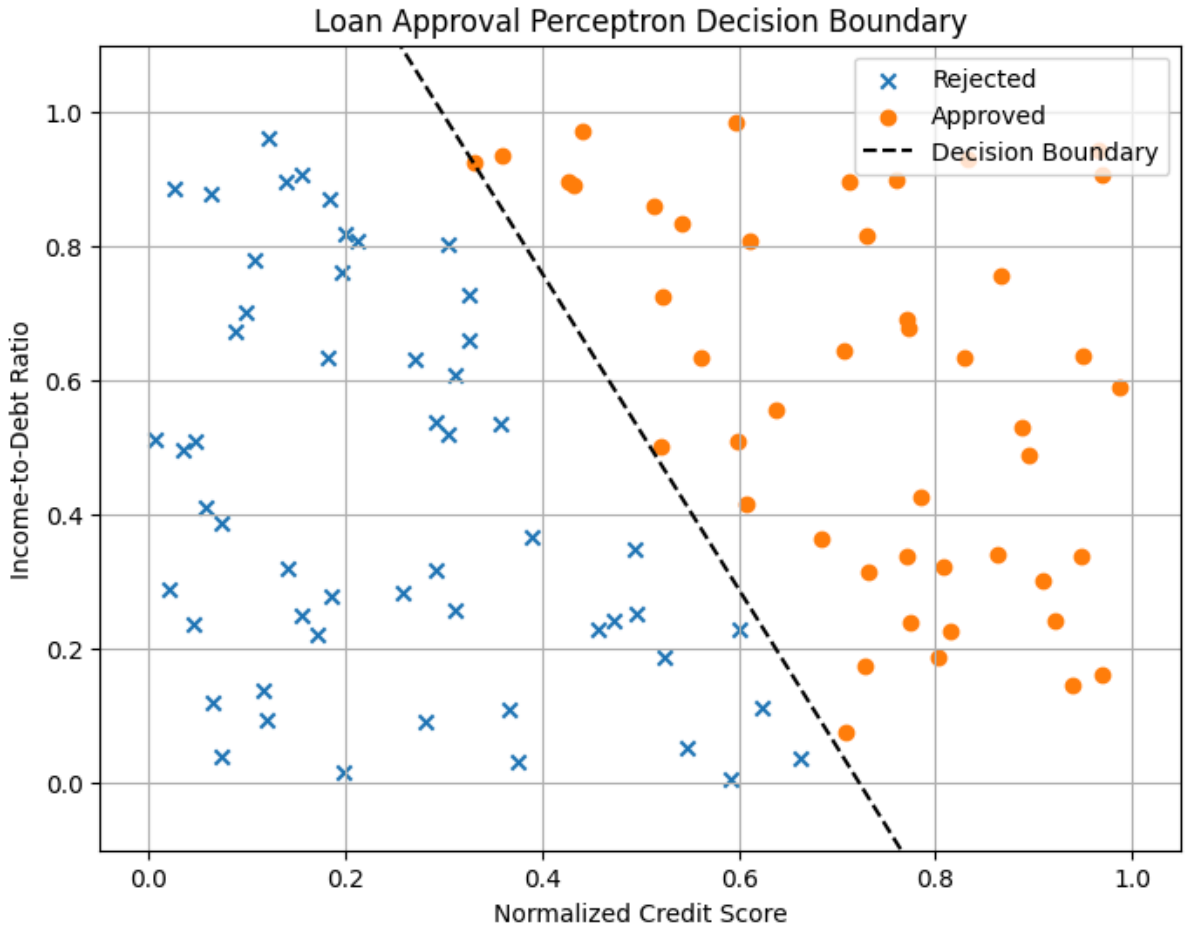
OUTPUT

```
Number of applications: 100
Approved: 46
Rejected: 54
Epoch  1: Weights = [0.1937 0.1566], Bias = -0.2000, Errors = 22
Epoch  2: Weights = [0.3033 0.2081], Bias = -0.3000, Errors = 17
Epoch  3: Weights = [0.4688 0.1632], Bias = -0.3000, Errors = 12
Epoch  4: Weights = [0.4701 0.3029], Bias = -0.3000, Errors = 10
Epoch  5: Weights = [0.5065 0.2631], Bias = -0.4000, Errors = 9
Epoch  6: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 4
Epoch  7: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch  8: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch  9: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 10: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 11: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 12: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 13: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 14: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 15: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 16: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 17: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 18: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 19: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0
Epoch 20: Weights = [0.5534 0.2351], Bias = -0.4000, Errors = 0

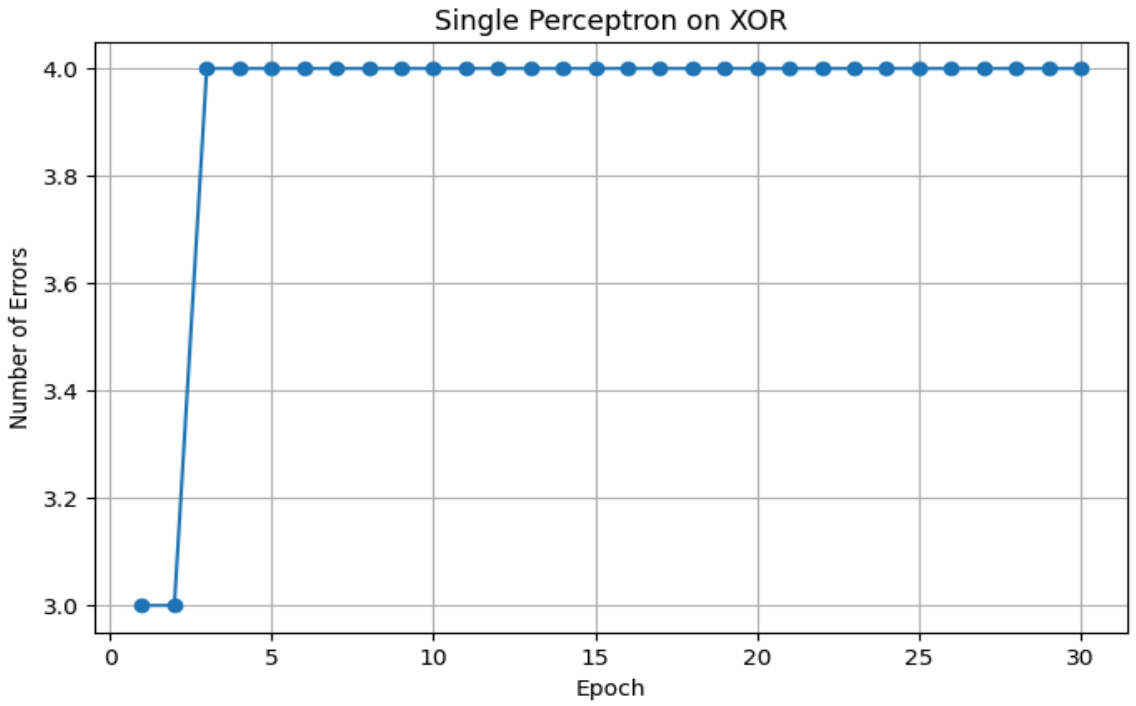
Final Weights: [0.55341172 0.23505902]
Final Bias: -0.4
Training Accuracy: 100.0 %
```

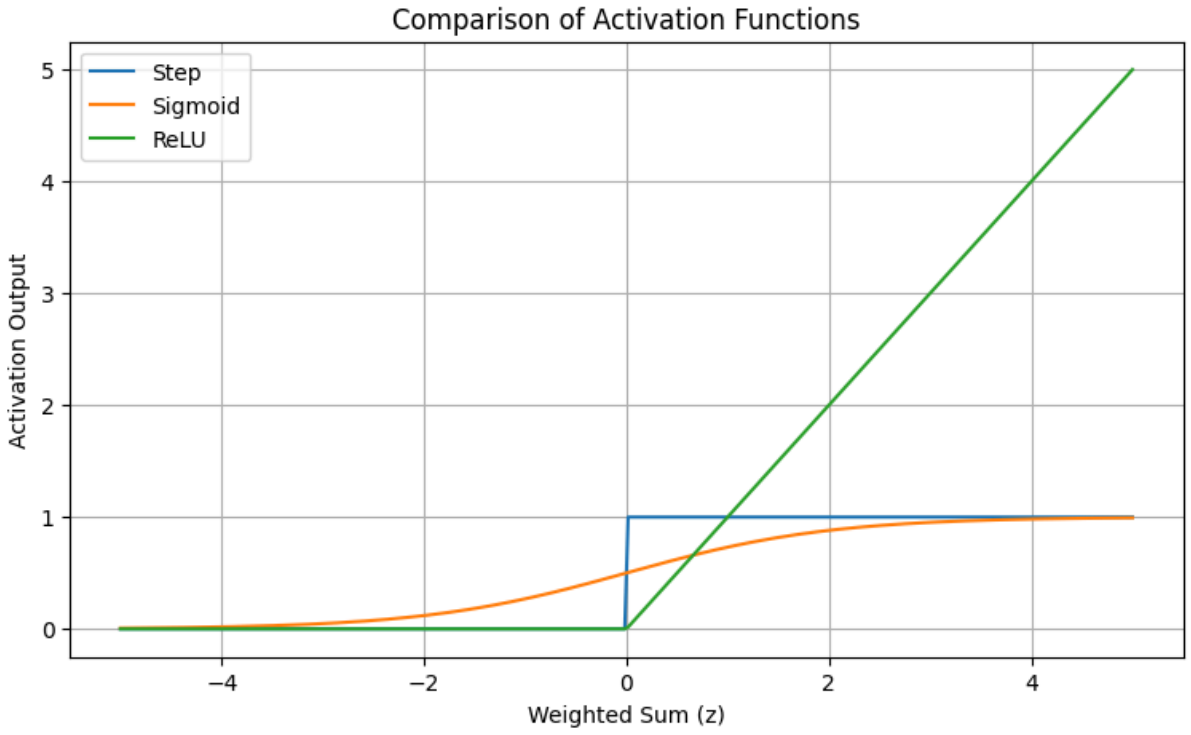

Perceptron Weight Updates
Weight 1
Weight 2
Weight Value
Epoch
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Perceptron Learning Progress
Number of Misclassifications
Epoch
20
15
10
5
0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Loan Approval Perceptron Decision Boundary
Rejected
Approved
Decision Boundary
Income-to-Debt Ratio
Normalized Credit Score
0.0
0.2
0.4
0.6
0.8
1.0

XOR Inputs:
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
Actual XOR Outputs: [0 1 1 0]
Perceptron Predictions: [1 1 0 0]
XOR Accuracy: 50.0 %





5. RESULT

A single-layer perceptron was implemented using NumPy to perform forward propagation and update weights using the perceptron learning rule. The model was trained on synthetic loan approval data, and its weight updates, training errors, and decision boundary were visualized. The XOR experiment demonstrated the limitation of a single perceptron for non-linearly separable data. Finally, step, sigmoid, and ReLU activation functions were compared graphically.